



Head-dump

head-dump 



Easy

Web Exploitation

picoCTF 2025

browser_webshell_solvable

AUTHOR: PRINCE NIYONSHUTI N.

Description

Welcome to the challenge! In this challenge, you will explore a web application and find an endpoint that exposes a file containing a hidden flag.

The application is a simple blog website where you can read articles about various topics, including an article about API Documentation. Your goal is to explore the application and find the endpoint that generates files holding the server's memory, where a secret flag is hidden.

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is:

NOT_RUNNING

Launch Instance

Hints ?

1

2

Explore backend development with us

20,545 users solved



75%



Liked

 picoCTF{FLAG}

Submit

Author: The Analyst: Hyposelenia

Challenge: Head-dump

Category: Web Exploitation

Date: 02/22/26



I. Objective

The objective of this challenge is to **identify and extract sensitive information exposed through an API endpoint**, specifically by accessing and analyzing a memory dump file in order to retrieve the hidden flag.

II. Background

Web applications often expose **API endpoints** to allow systems and users to interact with data. Some APIs are meant only for **developers or administrators**, but when they are publicly accessible, they can accidentally leak sensitive information.

A **heap dump** (or memory dump) is a snapshot of data stored in a program's memory while it is running. These dumps may contain:

- Credentials
- Tokens
- Debug information
- Sensitive strings (like flags)

In real-world cybersecurity incidents, exposed memory dumps have led to **serious data leaks**. This challenge demonstrates how **improperly secured API endpoints** can expose internal application data.

III. Tool Used

- **Web Browser** - Used to explore the challenge website and API documentation
- **Browser Developer Tools (Inspect)** - Used to analyze page structure
- **Kali Linux (Oracle VirtualBox)** - Used for terminal-based analysis



- **wget** - Used to download files from a URL
- **cat** - Used to read file contents
- **grep** - Used to filter file contents

IV. Methodology

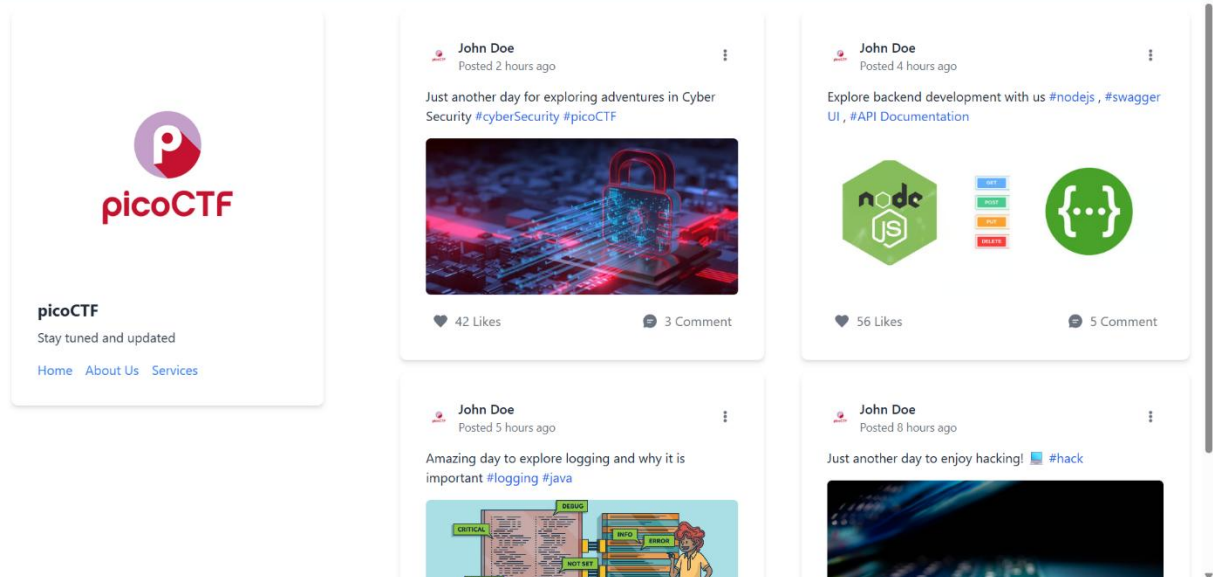
1. Launched the challenge instance

Its current status is:

NOT_RUNNING

Launch Instance

2. Accessed the provided URL link



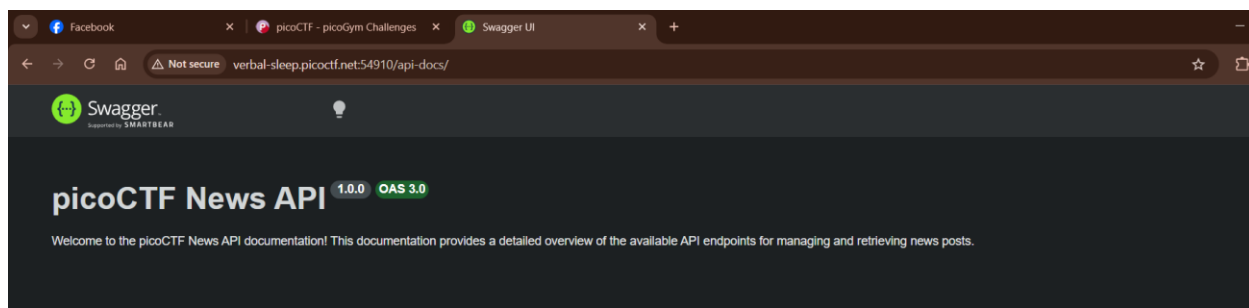


3. Explored the website and focused on the **API Documentation**, as hinted by the challenge description

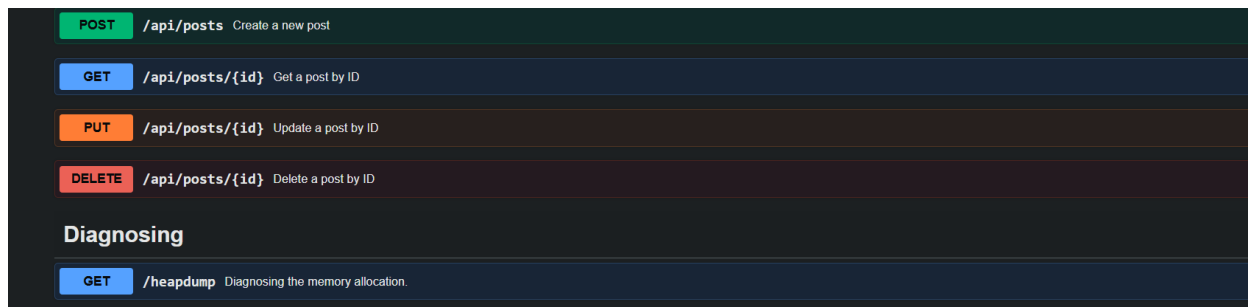
Explore backend development with us
UI , #API Documentation

There is also “Explore backend w/ us” just like in the hint.

4. Clicked **#API Documentation**, which opened the **Swagger UI**



5. Scrolled through the available endpoints and noticed a **HEAD/GET endpoint** related to a heap dump



6. Clicked **GET** → **Try it out** → **Execute**



Diagnosing

GET /heapdump Diagnosing the memory allocation.

Parameters

No parameters

Responses

Code	Description	Links
200	Returns a memory allocation status.	No links

Diagnosing

GET /heapdump Diagnosing the memory allocation.

Parameters

No parameters

Execute

Responses

Code	Description	Links
200	Returns a memory allocation status.	No links

7. After execution, a **request URL** was provided

Responses

Curl

```
curl -X 'GET' \
'http://verbal-sleep.picocftf.net:54910/heapdump' \
-H 'accept: */*'
```

Request URL

```
http://verbal-sleep.picocftf.net:54910/heapdump
```

Copy that Request URL

8. Switched to Kali Linux and used the `wget` command with the provided URL to download the file



```
File Actions Edit View Help
(kali@kali)-[/media/sf_Downloaded_Cybersecfiles]
$ wget http://verbal-sleep.picoctf.net:54910/heapdump
--2026-02-21 20:11:24-- http://verbal-sleep.picoctf.net:54910/heapdump
Resolving verbal-sleep.picoctf.net (verbal-sleep.picoctf.net)... 3.138.217.147
Connecting to verbal-sleep.picoctf.net (verbal-sleep.picoctf.net)|3.138.217.147|:54910... connected.
HTTP request sent, awaiting response... 200 OK
Length: 11263367 (11M) [application/octet-stream]
Saving to: 'heapdump'

heapdump 100%[=====] 10.74M 2.40MB/s in 4.5s
2026-02-21 20:11:30 (2.40 MB/s) - 'heapdump' saved [11263367/11263367]
```

Zoom in to read.

9. The downloaded file was identified as a **heap-dump**

```
heapdump 100%[=====]
2026-02-21 20:11:30 (2.40 MB/s) - 'heapdump' saved [11263367/11263367]

(kali@kali)-[/media/sf_Downloaded_Cybersecfiles]
$ ls
heapdump

(kali@kali)-[/media/sf_Downloaded_Cybersecfiles]
$
```

10.Used the `cat` command to read the file contents

11.Since memory dumps can be very large and messy, used `grep` to filter only useful data

12.Executed `cat filename | grep picoCTF` to search specifically for the flag



What is an API?



An API or **Application Programming Interface** is like a helper robot that lets two things talk to each other.

Think it like this:

Imagine you are in a classroom:

- You want a pencil.
- You don't go to the teacher's desk and open drawers.
- You **raise your hand and ask**.

The teacher gives you the pencil.

You asking = request

Teacher responding = API response

A website uses APIs to:

- Get data
- Send data
- Do actions **without showing how it works inside**

What does an API do?

An API:

- Accepts a request (asking for something)
- Processes it safely
- Sends back only what is allowed

Examples:

- Logging in
- Loading user profiles
- Showing posts or messages

APIs are meant to protect the system, not expose it.

What is an API Endpoint?

An API endpoint is like a specific door.

House analogy:

- Front door → public (homepage)
- Kitchen → family only (let's say it like that)



- Safe → owner only

Each door has an address.

An API endpoint is:

- A specific URL
- That does one specific job

Here, an example:

/api/getUser
/api/login
/api/heapdump

Problem:

Some doors are **not supposed to be public**, and one's safe must never be for public.

Why API endpoints must NOT be exposed

If a sensitive endpoint is public:

- Anyone can access it
- No permission is required
- Private data leaks

In this challenge:

- The **heap dump endpoint was public**
- No login or protection was required
- Anyone could download internal memory data

That is **very dangerous in real life**.

What does "HEAD" mean?



In HTTP, **HEAD** is a request type.

Simple explanation:

- **GET** = “Give me the thing”
- **HEAD** = “Tell me about the thing, but don’t show it”

HEAD usually tells:

- File size
- File type
- Whether a file exists

In this challenge:

- HEAD helped **identify the heap dump**
- GET allowed **downloading the file**

What is a Heap Dump?

A heap dump is like:

A notebook that writes down everything a program is thinking.

It may contain:

- Passwords
- Tokens
- Debug data
- Secrets
- Flags

Heap dumps are meant for:

- Developers
- Debugging
- Internal use only



They must NEVER be public.

How do we know a file can be downloaded with wget?

A file can be downloaded if:

- The **server responds with a file**
- It does **not require browser-only interaction**
- The URL directly serves content

Difference between a webpage and downloadable file:

Web Page	Downloadable File
HTML rendered	Raw file
Needs browser	Can use terminal
Interactive	Direct response

Since the endpoint returned a file:

- wget could download it directly

Why we used cat, |, and grep?

cat

Reads a file and prints its contents.

Think: “Open the notebook and read everything.”

Pipe “|”

Passes output from one command to another.

Think: “Read the notebook → hand important lines to someone else.”



grep

Searches for specific words.

Think: “Only show lines with **picoCTF**.”

Why we used them together:

cat heapdump | grep picoCTF

This:

- Reads the entire memory dump
- Filters out noise
- Finds the flag quickly

What was wrong in this challenge?

The website:

1. Exposed internal API documentation
2. Left a sensitive endpoint publicly accessible
3. Allowed downloading a heap dump without authentication
4. Stored sensitive data in memory without protection

How this should be prevented?

To prevent this:

- Disable debugging endpoints in production
- Protect APIs with authentication
- Restrict access by role (admin only)
- Never expose memory dumps publicly
- Remove API documentation from live sites



Key Lesson

Just because something works does not mean it is safe.

This challenge teaches that:

- APIs are powerful
- But dangerous when misconfigured
- Small mistakes can cause huge leaks

Okay, that is all... 'til next time! Toodle~doo

VII. Conclusion

The Head-dump challenge highlights the **dangers of exposing internal debugging tools through public APIs.**

Through this challenge, I learned that:

- API documentation can reveal sensitive endpoints
- Heap dumps may contain confidential data
- Command-line filtering tools are essential for analysis

This reinforces an important cybersecurity principle:

Debugging features must never be exposed in production environments.

— The Analyst: Hyposelenia